

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e3b53aed-f4e\agent-aca5076b773447c45.jsonl`
- SHA-256: `61117884629ffae55646eba12b3c14b072d38ed027f71a1fc40683f691c10372`
- Source modified: `2026-06-18T02:08:45+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_e3b53aed-f4e`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T02:04:19.551Z)

You are the merge gatekeeper for pr213-scripts-unwrap. Adversarial reviews (JSON):

```
[
  {
    "lens": "correctness",
    "blocks_merge": false,
    "findings": [
      {
        "severity": "none",
        "location": "backend/results.py",
        "issue": "No defect found.",
        "suggestion": "Merge."
      }
    ],
    "summary": "Fix is correct. Scripts crashed on len of a raw Success or Failure dataclass under the default motion segmenter. unwrap_or_failure mirrors the backend main inline unwrap and handles Success, Failure, and bare list. Failure branches correct: continue in the batch driver, return in the single-video script. Full suite 850 passed, 7 skipped. New tests 4 passed. Ruff and mypy clean. No must-fix issues.</summary>\n</invoke>\n"
  },
  {
    "lens": "edge cases: partial states, None/empty, mixed inputs, exception types (OperationalError text stability / different OperationalError), empty Success(value=[]) flowing to the empty-guard",
    "summary": "Real, correct bug fix. Confirmed the premise: the default `motion` segmenter (backend/pipeline/segmenters/motion.py:224,31) returns Success/Failure, so the old raw len()/truthiness on a Success dataclass genuinely crashed with TypeError. The new backend/results.py:63 unwrap_or_failure faithfully mirrors the inline unwrap at backend/main.py:201,213, both script call sites (scripts/run_process_and_stitch.py:85, scripts/run_phase3_eval.py:149) are correctly converted with proper failure branching, and the new tests/test_results.py passes 4/4. I verified every lens edge case at runtime: Failure -> (None, f); empty Success(value=[]) -> ([], None) which flows to the empty-guard as a list exactly as intended; bare list passes through. The OperationalError lens concern does not apply ? the sole handler (backend/infrastructure/database.py:40) is a broad `except sqlite3.OperationalError: pass` with no message-text matching and is untouched by this PR. Ruff clean; mypy clean on backend/results.py (CI runs `mypy backend training`, ci.yml:48, which does NOT include scripts/). The one full-suite failure (test_fusion_hybrid::test_gate_a_never_empties_a_video) is a pre-existing Windows test-isolation artifact (passes in isolation; FileNotFoundError from a leftover output/chunks_gemini_handoff dir) in code this PR does not touch. No must-fix issues; safe to merge.",
    "findings": [
      {
        "severity": "nit",
        "location": "tools/generate_highlights.py:167-169",
```

```

        "issue": "A fourth hand-rolled copy of the same Success/Failure unwrap
(isinstance(result, Failure) ... ) now exists alongside backend/main.py:201,213 and the two
scripts this PR converted. It is correct (and reads segments back from the DB so it never needs
.value), so it is not buggy and is arguably out of this PR's scope, but it is the exact DRY
target unwrap_or_failure was introduced to eliminate.",
        "suggestion": "Optionally fold tools/generate_highlights.py and the backend/main.py
inline unwrap onto unwrap_or_failure in a follow-up so there is a single unwrap implementation."
    },
    {
        "severity": "nit",
        "location": "scripts/run_phase3_eval.py:155",
        "issue": "mypy flags `len(segs)` as `len(Any | None)` because unwrap_or_failure returns
Tuple[Optional[Any], Optional[Failure]]. This is informational only: CI runs `mypy backend
training` (.github/workflows/ci.yml:48) and does not type-check scripts/, and the master version
(len() on a raw Success|Failure Result) would have been equally untyped. The sibling errors at
lines 106 and 150 are pre-existing AppConfig/Optional-segmenter patterns unrelated to this
change.",
        "suggestion": "No action required for merge. If scripts/ is ever added to the mypy gate,
narrow the helper's value type or add a local assert/cast before len()."
    }
],
"blocks_merge": false
},
{
    "blocks_merge": false,
    "findings": [
        {
            "severity": "nit",
            "location": "scripts/run_process_and_stitch.py:82-84",
            "issue": "Comment says process_video crashes under the default motion segmenter, but
this script's actual default is cfg.indexing.default_segmenter = yolo_hybrid
(backend/config/models.py:118), which returns bare lists and never crashed. Crash only occurs
when a user configures motion. Fix is correct regardless; comment premise is slightly off for
this script (run_phase3_eval.py genuinely defaults to motion at line 54).",
            "suggestion": "Reword the comment to reference the configured segmenter rather than
implying motion is this script's default."
        },
        {
            "severity": "nit",
            "location": "tests/test_results.py:21-27",
            "issue": "No test directly pins unwrap_or_failure(Success(value=[])) -> ([], None), the
empty-success path run_process_and_stitch.py's empty-guard depends on.",
            "suggestion": "Add a case asserting unwrap_or_failure(Success(value=[])) returns ([,
None).",
        }
    ],
    "lens": "test quality",
    "summary": "PR 213 correctly fixes a real TypeError: the default motion segmenter returns a
Success/Failure dataclass (motion.py:224 and :31), and the two scripts previously called
len/truthiness on it directly. The new backend.results.unwrap_or_failure is a faithful, verified
DRY of the inline unwrap in backend/main.py:201-213, and passes bare-list legacy segmenters
(yolo_hybrid) through unchanged. Test-quality lens: tests/test_results.py is deterministic and
fully offline (pure dataclass import, ran in 0.10s), auto-discovered by run_all_tests.py and
pytest, and all 4 tests pass. The key regression test genuinely pins the fix - it asserts len on
a Success raises TypeError and that the helper yields a len-able list, so it would fail if the
helper reverted to returning the raw Success. Assertions are meaningful (identity, value
equality, length). No must-fix issues; two nits.</parameter>\n</invoke>\n"
},
{
    "lens": "scope & convention",
    "summary": "PR #213 fixes a real, verifiable crash: scripts/run_phase3_eval.py and
scripts/run_process_and_stitch.py called len()/truthiness directly on the raw segmenter Result,
and motion.py (segmenters/motion.py:31,224) returns Failure/Success dataclasses that have no
__len__ -> TypeError. The new backend/results.py:unwrap_or_failure faithfully DRYs the exact
inline unwrap already in backend/main.py (lines 201/213 and 838/843): isinstance(Failure) then

```

```

`result.value if hasattr(result,'value') else result`. Both script edits integrate correctly
(the cache-hit segments=[] init path in run_process_and_stitch.py stays safe), and adding a
helper to backend/results.py is non-disruptive ? main.py's two inline sites are untouched and
still pass. Scope is exactly right: only the 4 expected files changed, single clean commit off
origin/master, no behavior/product/licensing change. Verified locally: full suite 850 passed / 7
skipped, new tests/test_results.py passes, ruff and mypy clean on all touched files. No other
caller has the same latent bug (tools/generate_highlights.py:168 already handles Failure and
reads segments from the DB). Edge cases (Success(value=None), dict with a 'value' key, bare
list) all behave correctly. Only findings are minor prose-accuracy nits, no code defects.
Recommend merge.",
    "findings": [
        {
            "severity": "nit",
            "location": "tests/test_results.py:32-33",
            "issue": "The test docstring says callers 'run_phase3_eval / run_process_and_stitch ...
default to the motion segmenter'. This is accurate for run_phase3_eval.py (its --segmenter
argparse default is literally 'motion', scripts/run_phase3_eval.py:54), but imprecise for
run_process_and_stitch.py, which uses cfg.indexing.default_segmenter ? and the config default is
'yolo_hybrid' (backend/config/models.py:118), a segmenter that returns a bare list and would NOT
have crashed under default config. motion is opt-in there. The fix is still correct and forward-
safe regardless; only the prose overstates the default-path scope for the second script.",
            "suggestion": "Optionally reword to 'run_phase3_eval defaults to motion;
run_process_and_stitch crashes when configured with a Result-returning segmenter like motion' to
keep the docstring strictly accurate. Non-blocking."
        },
        {
            "severity": "nit",
            "location": "scripts/run_process_and_stitch.py:82-84 (and commit subject)",
            "issue": "The commit subject 'motion crashes the default path' and the inline comment
frame motion as the default for this script too. Under default config this script runs
yolo_hybrid (bare list, no crash); the crash here is latent (only when default_segmenter is set
to motion or another Result-returning segmenter). The comment's own wording 'for some segmenters
(e.g. motion)' is actually correct ? it's just the commit subject that generalizes.",
            "suggestion": "No code change needed. The fix is correct for both scripts; the
imprecision is only in the commit subject's 'default path' phrasing. Mention in PR description
that the run_process_and_stitch crash is latent-under-non-default-config if desired."
        },
        {
            "severity": "nit",
            "location": "tools/generate_highlights.py:167-171",
            "issue": "A third production caller does the same isinstance(result, Failure) unwrap
inline. It is correct (handles Failure and reads segments back from the DB, so no len() crash),
so it is rightly out of scope for this bug fix ? but it is a candidate to also adopt
unwrap_or_failure for full DRY consistency.",
            "suggestion": "Optional follow-up: migrate generate_highlights.py and main.py's two
inline sites onto unwrap_or_failure in a later cleanup PR. Do not expand this PR's scope."
        }
    ],
    "blocks_merge": false
}
]

```

Return verdict="merge" only if NO genuine must-fix exists. Dedup across lenses; be skeptical of non-issues. List real must_fixes (empty if none) and should_fixes with a crisp rationale. Set pr="pr213-scripts-unwrap".

2. assistant (2026-06-18T02:04:22.850Z)

I'll verify the key claims in these reviews before signing off. Let me start by pulling the latest and examining the actual code.

3. user (2026-06-18T02:04:26.616Z)

---current---
docs/golden-regression-fixtures-plan

---log---

f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)
8c68143 feat(eval): R1 ? net over-seg promotion guard unblocks the cap6+R4 milestone (#201)
2eaccd5 feat(eval): nightly rally-quality regression tripwire (golden corpus, CPU) (#202)
34b9caa feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding
(#200)
7e8f88f feat(eval): R2 ? task-faithful tolerance-match rally metric (augments tIoU) (#199)

4. assistant (2026-06-18T02:04:29.463Z)

The PR branch isn't fetched locally. Let me find it and check the actual diff.

5. user (2026-06-18T02:04:36.762Z)

```
+ bhi-execpolicy
* docs/golden-regression-fixtures-plan
+ docs/golden-videos-tracker
+ docs/quality-iterations-realfootage
+ docs/rally-gap-closure
+ docs/rally-quality-research
  feat/highlights-runner
+ feat/local-no-ai-segmenter
  feat/r1-milestone-config-flip
  feat/r1-net-overseg-guard
+ feat/r2-tolerance-metric
+ feat/r4-inactivity-end
  feat/r5-blob-splitter
+ feat/tier0-rally-seg-eval
+ feat/w1-hard-cap-fallback
+ feat/w2-netcross-gate
+ feat/w2-never-empty-safeguard
  fix/compiler-dedup-overlapping-segments
+ fix/db-migration-idempotent-alters
  fix/fusion-loaders-mixed-schema
  fix/gemini-per-video-chunk-dir
+ fix/runner-stitch-watermark
+ fix/scripts-unwrap-segmenter-result
  fix/wasb-test-mocks-readability-preflight
  fix/wsl-unreadable-source-preflight
+ master
+ perf/wasb-fast-frames
+ run/boxhill17-highlights
worktree-agent-a771ff1d85e18f2ed
remotes/origin/HEAD -> origin/master
remotes/origin/claude/indexer-doc-review-XVPT
remotes/origin/docs/golden-regression-fixtures-plan
remotes/origin/docs/golden-videos-tracker
remotes/origin/docs/pmf-field-signals
remotes/origin/docs/quality-iterations-realfootage
remotes/origin/docs/r2-eval-metric
remotes/origin/docs/rally-gap-closure
remotes/origin/docs/rally-quality-research
remotes/origin/feat/golden-fixtures-audit-hardening
remotes/origin/feat/golden-fixtures-crossos
remotes/origin/feat/golden-fixtures-doc
remotes/origin/feat/golden-fixtures-m1
remotes/origin/feat/golden-fixtures-p1
remotes/origin/feat/highlights-runner
remotes/origin/feat/local-no-ai-segmenter
remotes/origin/feat/nightly-regression
remotes/origin/feat/r1-milestone-config-flip
remotes/origin/feat/r1-net-overseg-guard
remotes/origin/feat/r2-tolerance-metric
remotes/origin/feat/r4-inactivity-end
remotes/origin/feat/r5-blob-splitter
```

```
remotes/origin/feat/tier0-rally-seg-eval
remotes/origin/feat/tier1-bounded-windowing
remotes/origin/feat/w1-hard-cap-fallback
remotes/origin/feat/w2-netcross-gate
remotes/origin/feat/w2-never-empty-safeguard
remotes/origin/fix/ablation-litmus-mixed-schema
remotes/origin/fix/compiler-dedup-overlapping-segments
remotes/origin/fix/db-migration-idempotent-alters
remotes/origin/fix/fusion-loaders-mixed-schema
remotes/origin/fix/gemini-per-video-chunk-dir
remotes/origin/fix/runner-stitch-watermark
remotes/origin/fix/scripts-unwrap-segmenter-result
remotes/origin/fix/wasb-test-mocks-readability-preflight
remotes/origin/fix/wsl-unreadable-source-preflight
remotes/origin/master
remotes/origin/perf/wasb-fast-frames
==REMOTES==
==STASH/OTHER==
03d11d9 fix(scripts): unwrap the segmenter Result before len()/truthiness (motion crashes the
default path)
11e1c23 test: exercise is_success() and unwrap() on Success/Failure to reach 100% coverage on
results.py
```

6. assistant (2026-06-18T02:04:39.348Z)

The branch is `fix/scripts-unwrap-segmenter-result`. Let me examine the diff against master and the key files.

7. user (2026-06-18T02:04:40.496Z)

```
backend/results.py | 20 ++++++-----
scripts/run_phase3_eval.py | 11 ++++++--
scripts/run_process_and_stitch.py | 9 ++++++
tests/test_results.py | 39 ++++++-----
4 files changed, 76 insertions(+), 3 deletions(-)
=====FULL DIFF=====
diff --git a/backend/results.py b/backend/results.py
index d6b1ea4..75120c5 100644
--- a/backend/results.py
+++ b/backend/results.py
@@ -18,7 +18,7 @@ This module has no dependencies on the rest of the app.
 from __future__ import annotations

 from dataclasses import dataclass, field
-from typing import Generic, TypeVar, List, Optional, Any, Union
+from typing import Generic, TypeVar, List, Optional, Any, Tuple, Union

 T = TypeVar("T")

@@ -58,3 +58,21 @@ class Failure:
 # Result[ValueType] means Success[ValueType] | Failure
 # (the Failure "error" type is fixed for this contract)
 Result = Union[Success[T], Failure]
+
+
+def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
+    """Split a segmenter/provider return into ``(value, failure)`` so a caller branches once.
+
+    - ``Failure`` -> ``(None, failure)``
+    - ``Success(value=v)`` -> ``(v, None)``
+    - a bare value (legacy code that predates this contract and still returns a raw list)
+    -> ``(value, None)``
+
+    This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline; callers
+    that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
```

```

len()``
+   on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
+   (Success | Failure | legacy bare value)."""
+   if isinstance(result, Failure):
+       return None, result
+   value = result.value if hasattr(result, "value") else result
+   return value, None
diff --git a/scripts/run_phase3_eval.py b/scripts/run_phase3_eval.py
index 56e21ca..dd1263e 100644
--- a/scripts/run_phase3_eval.py
+++ b/scripts/run_phase3_eval.py
@@ -34,6 +34,7 @@ from backend.config import load_config
    from backend.utils.hashing import compute_video_id as get_file_md5 # canonical video_id
    # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
    from backend.pipeline.segmenters import segmenter_registry
+from backend.results import unwrap_or_failure
    from backend.eval.annotations import load_annotations
    from backend.eval.harness import evaluate, format_report_text, report_to_dict
    from backend.eval import batch
@@ -142,7 +143,15 @@ def main(argv=None) -> int:
    print(f"[proc] {vid}: segmenting with {args.segmenter}...")
    db.add_video(video_id, video_path, "processed", [])
    t0 = time.time()
-   segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
+   # process_video returns a Result (Success|Failure) for some segmenters
+   # (e.g. the default `motion`); unwrap before len() so this doesn't crash with
+   # TypeError on a Success dataclass ? mirrors the unwrap in backend/main.py.
+   segs, failure = unwrap_or_failure(
+       segmenter.process_video(video_id, video_path, max_frames=args.max_frames))
+   if failure is not None:
+       print(f"        -> FAILED: {failure.message}")
+       skipped.append((vid, f"segment failed: {failure.message}"))
+       continue
    print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")

    gt = load_annotations(csv_path)
diff --git a/scripts/run_process_and_stitch.py b/scripts/run_process_and_stitch.py
index b8ffef6..ea7c2de 100644
--- a/scripts/run_process_and_stitch.py
+++ b/scripts/run_process_and_stitch.py
@@ -16,6 +16,7 @@ from backend.config import load_config
    from backend.utils.hashing import compute_video_id
    from backend.pipeline.segmenters import segmenter_registry
    from backend.pipeline.stitcher.compiler import VideoCompiler
+from backend.results import unwrap_or_failure
    from backend.tools.export import format_rally_summary

    logger = logging.getLogger(__name__)
@@ -78,7 +79,13 @@ def main():
    return
    print(f"Using segmenter: {seg_name}", flush=True)
    segmenter = segmenter_cls(db, cfg)
-   segments = segmenter.process_video(video_id, video_path)
+   # process_video returns a Result (Success|Failure) for some segmenters (e.g. `motion`);
+   # unwrap before len()/truthiness so this doesn't crash with TypeError on a Success
+   # dataclass (and so the empty-guard below actually sees the list) ? mirrors
backend/main.py.
+   segments, failure = unwrap_or_failure(segmenter.process_video(video_id, video_path))
+   if failure is not None:
+       print(f"[FAIL] Segmentation failed: {failure.message}")
+       return
    print(f"Successfully indexed {len(segments)} play segments.", flush=True)

```

```

    if not segments:
diff --git a/tests/test_results.py b/tests/test_results.py
new file mode 100644
index 0000000..5b50767
--- /dev/null
+++ b/tests/test_results.py
@@ -0,0 +1,39 @@
+"""Unit tests for ``backend.results`` ? the Success/Failure contract + the
+``unwrap_or_failure``
+helper that standalone scripts use to avoid ``len()``-ing a raw ``Success`` dataclass."""
+import pytest
+
+
+from backend.results import Failure, Success, unwrap_or_failure
+
+
+def test_unwrap_success_returns_value_and_no_failure():
+    value, failure = unwrap_or_failure(Success(value=[{"start": 1.0}]))
+    assert value == [{"start": 1.0}]
+    assert failure is None
+
+
+def test_unwrap_failure_returns_failure_and_no_value():
+    f = Failure(message="Could not open video: x.mp4", is_recoverable=False)
+    value, failure = unwrap_or_failure(f)
+    assert value is None
+    assert failure is f
+
+
+def test_unwrap_bare_value_passes_through():
+    """Legacy segmenters that predate the Result contract still return a bare list ? the helper
+    passes those through unchanged as ``(value, None)`` so a mixed segmenter set all works."""
+    bare = [{"start": 1.0}, {"start": 2.0}]
+    value, failure = unwrap_or_failure(bare)
+    assert value == bare
+    assert failure is None
+
+
+def test_unwrap_fixes_the_len_on_success_crash():
+    """Regression for the scripts bug: ``len()`` on a raw ``Success`` raises ``TypeError``;
+    unwrapping first yields the underlying list so callers (run_phase3_eval /
+    run_process_and_stitch, which default to the ``motion`` segmenter) can ``len()`` it."""
+    result = Success(value=[1, 2, 3])
+    with pytest.raises(TypeError):
+        len(result)
+    # the original bug (Success has no __len__)
+    segs, failure = unwrap_or_failure(result)
+    assert failure is None
+    assert len(segs) == 3
+    # fixed

```

8. assistant (2026-06-18T02:04:43.481Z)

Now let me verify the core premises. First, the bug premise: does `motion.py` actually return Success/Failure, and what is the actual default for each script.

9. user (2026-06-18T02:04:44.916Z)

=DONE=

10. user (2026-06-18T02:04:48.553Z)

```

===== motion.py return type (lines around 31, 224) =====
    if not player_pool:
        player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]

    cap = cv2.VideoCapture(video_path)

```

```

if not cap.isOpened():
    logger.error(f"Segmenter could not open video: {video_path}")
    return Failure(message=f"Could not open video: {video_path}", is_recoverable=False)

fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
duration = total_frames / fps if fps > 0 else 0

# Sub-sample settings (5 FPS for analysis)
analysis_fps = 5.0
frame_interval = max(1, int(fps / analysis_fps))

        "id": segment_id,
        "start_time": start,
        "end_time": end,
        "duration": dur,
        "server": server,
        "receiver": receiver,
        "metadata": metadata
    })

return Success(value=segments_data)

```

Register the segmenter

```

segmenter_registry.register(MotionPlaySegmenter)
===== main.py inline unwrap (201,213) =====
    # state (same destroy-after-confirm contract as the Failure branch), then let
    # the job worker record the failure.
    db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
    raise
segmentation_ran = True

if isinstance(result, Failure):
    logger.error(f"[API] Segmenter failed: {result.message}")
    db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
    response = {
        "video_id": video_id,
        "status": "failed",
        "message": f"Segmenter '{seg_name}' failed: {result.message}",
        "results": [r.model_dump() for r in val_results],
        "play_segments": [],
    }
    return response

segments = result.value if hasattr(result, "value") else result
notify("finalizing")
_finalize_reingest(video_id, segments, play_wm, cand_wm)

response = {
    "video_id": video_id,
    "status": "success",
    "message": f"Successfully indexed {len(segments)} play segments using '{seg_name}'
segmenter.",
    "results": [r.model_dump() for r in val_results],
    "play_segments": segments,
}
return response
finally:

```

11. assistant (2026-06-18T02:04:51.956Z)

Both premises confirmed: motion.py returns `Failure`/`Success`, and the helper faithfully mirrors `backend/main.py`'s inline `isinstance(result, Failure)` + `result.value if hasattr(result, "value") else result`. Now the default-segmenter claim ? the reviews disagree

about whether `run\_process\_and\_stitch.py` defaults to motion.

12. user (2026-06-18T02:04:54.082Z)

```
==== run_phase3_eval.py --segmenter default (line ~54) ====
    formatter_class=argparse.RawDescriptionHelpFormatter,
)
p.add_argument("--manifest", required=True, help="Path to the collector's manifest.json.")
p.add_argument("--videos-root", default=None,
                help="Root for resolving manifest relative local_paths "
                    "(default: the collector repo root, two levels above the manifest).")
p.add_argument("--segmenter", default="motion",
                help="Segmenter to run (default: motion ? free/CPU). "
                    "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.")
p.add_argument("--stage", choices=("candidates", "final", "both", "auto"), default="auto",
                help="Which prediction stage(s) to score (default: auto by segmenter).")
p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default: 0.5).")
==== run_process_and_stitch.py seg_name resolution (~70-90) ====
    print("[CACHE] Using cached play segments. Skipping segmenter API call.",
flush=True)
        cache_hit = True
    else:
        print("[CACHE] Ignoring cache...", flush=True)
    else:
        print("[CACHE] Video exists in DB but contains 0 play segments. Treating as cache
miss.", flush=True)

    if not cache_hit:
        print("[CACHE] Initializing clean indexing...", flush=True)
        db.clear_video_data(video_id)

        print("Bypassing OpenCV validations for performance on massive files...", flush=True)
        db.add_video(video_id, video_path, "processed", [])

        print("Running segmenter & indexing...", flush=True)
        seg_name = cfg.indexing.default_segmenter
        segmenter_cls = segmenter_registry.get(seg_name)
        if not segmenter_cls:
            print(f"[FAIL] Segmenter '{seg_name}' not found.")
            return
        print(f"Using segmenter: {seg_name}", flush=True)
        segmenter = segmenter_cls(db, cfg)
        # process_video returns a Result (Success|Failure) for some segmenters (e.g. `motion`);
        # unwrap before len()/truthiness so this doesn't crash with TypeError on a Success
        # dataclass (and so the empty-guard below actually sees the list) ? mirrors
backend/main.py.
        segments, failure = unwrap_or_failure(segmenter.process_video(video_id, video_path))
        if failure is not None:
            print(f"[FAIL] Segmentation failed: {failure.message}")
            return
        print(f"Successfully indexed {len(segments)} play segments.", flush=True)

    if not segments:
        print("No play segments detected. Cannot compile highlights.")
        return

==== config default_segmenter (models.py ~118) ====
# net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
net_axis: Literal["x", "y"] = "y"
net_position: float = Field(default=0.5, ge=0.0, le=1.0)
```

```

class IndexingConfig(BaseModel):
    default_segmenter: str = "yolo_hybrid"
    use_gpu: bool = True
    motion_threshold: float = 0.08
    min_segment_duration: float = 3.0
    dead_time_merge_gap: float = 2.0

```

13. assistant (2026-06-18T02:05:02.119Z)

Confirmed the nuance the reviewers flagged:

- ``run_phase3_eval.py`` `--segmenter`` default is literally ``"motion"`` ? genuinely crashed under default.
- ``run_process_and_stitch.py`` uses ``cfg.indexing.default_segmenter`` whose config default is ``"yolo_hybrid"`` (returns a bare list, no crash). The crash there is latent ? only under non-default ``motion`` config. The inline comment correctly says "for some segmenters (e.g. ``motion``)", but the commit subject "motion crashes the default path" overgeneralizes. This is a prose nit, not a code defect.

The empty-guard claim: ``Success(value=[])`` ? ``([], None)`` flows into ``if not segments:`` which the code at line ~91 handles. Confirmed. Now let me check the other callers (tools/generate_highlights.py) and the second main.py site (838/843).

14. user (2026-06-18T02:05:02.758Z)

```

===== generate_highlights.py unwrap (~165-172) =====
    if detect_src != src:
        logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
                    os.path.basename(detect_src), name)
    logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

    # AppConfig is consumed by segmenters via its dict-compatible .get shim (same as
    # main.py); cast keeps the type checker honest about that contract.
    result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_detect)
    if isinstance(result, Failure):
        logger.error("segmentation FAILED: %s", getattr(result, "message", result))
        summary["failed"].append(name)
        continue

    segs = db.query_play_segments(vid, {})
    pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
    logger.info("rallies detected: %d", len(pairs))
===== main.py second site (~835-845) =====

    print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
    segmenter_actual = seg_name
    play_wm, cand_wm = db_instance.segment_watermarks(video_id)
    segmenter = segmenter_cls(db_instance, global_config)
    result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
    segmentation_ran = True

    if isinstance(result, Failure):
        print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
        print("[FAIL] Indexing halted for video.")
        segments = []
    else:
        segments = result.value if hasattr(result, "value") else result
        print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments inside
database: {db_path}")
        # Destroy-AFTER-confirm: keep new on success, restore prior on empty/failure.
        _finalize_reingest(video_id, segments, play_wm, cand_wm)
    finally:
        # Always emit the per-video observability report. Never raises.
===== any other raw process_video callers? =====
scripts/run_phase3_eval.py:146:          # process_video returns a Result

```

```

(Success|Failure) for some segmenters
scripts/run_phase3_eval.py:150:         segmenter.process_video(video_id, video_path,
max_frames=args.max_frames))
scripts/run_process_and_stitch.py:82:         # process_video returns a Result (Success|Failure)
for some segmenters (e.g. `motion`);
scripts/run_process_and_stitch.py:85:         segments, failure =
unwrap_or_failure(segmenter.process_video(video_id, video_path))
tools/generate_highlights.py:167:         result = seg_cls(db, cast(Dict[str, Any],
cfg)).process_video(vid, proc_detect)
backend/eval/served_gate_a.py:13: verification reflects what ``process_video`` actually hands to
Gemini.
backend/eval/served_gate_a.py:36: # TrajectoryHybridSegmenter.process_video reads for the served
fusion run).
backend/eval/served_gate_a.py:76:     # call the pure decision seams ? no process_video, no DB,
no provider ? exactly as the
backend/eval/windowing.py:7: (``TrajectoryHybridSegmenter.process_video``) windows COARSELY
(``vt=0.02, merge_gap=2.0,
backend/eval/windowing.py:105:     ``TrajectoryHybridSegmenter.process_video`` reads exactly
these three:
backend/eval/windowing.py:149:     Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob
reads exactly, against an
backend/main.py:192:         result = segmenter.process_video(video_id, req.video_path,
req.player_pool, req.max_frames)
backend/main.py:835:         result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
backend/results.py:72:     that forget it (e.g. ``len(process_video(...))``) crash with
``TypeError: ... has no len()``

```

15. assistant (2026-06-18T02:05:07.555Z)

All callers verified:

- `tools/generate\_highlights.py:168` ? handles `Failure`, then reads segments from DB
(`db.query\_play\_segments`), never calls `.value` or `len()` on the raw result. Correct, no
latent bug. It IS a 4th hand-rolled unwrap (the DRY nit the reviewers raised), but out of scope.
- Both `backend/main.py` sites ? untouched, still correct.
- No other raw `process\_video` caller does `len()`/truthiness on the result. The fix is
complete.

Now let me run the verification suite per the pre-LGTM gate: new tests, ruff, mypy, and the full
suite. Running the independent checks in parallel.

16. user (2026-06-18T02:05:09.569Z)

```

.... [100%]
4 passed in 0.08s

```

17. user (2026-06-18T02:05:09.823Z)

All checks passed!

18. user (2026-06-18T02:05:17.670Z)

```

backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 104 source files
===EXIT 0===

```

19. user (2026-06-18T02:05:17.897Z)

```
33: # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
35: # up. Typed core deps are installed so mypy sees their real signatures
45:     - name: Install mypy + typed core deps
46:       run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-genai
numpy
48:       run: mypy backend training
87:       # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
```

20. assistant (2026-06-18T02:05:21.751Z)

mypy clean on `backend training` (the CI scope, exit 0). CI does NOT type-check `scripts/`, so the `len(Any | None)` informational mypy note in `run\_phase3\_eval.py` is not gated. Now the full suite with coverage to confirm green and no coverage regression.

21. user (2026-06-18T02:05:22.684Z)

```
.github/workflows/ci.yml:97:         run: python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70
run_all_tests.py:10:     python run_all_tests.py --cov=backend --cov-report=term-missing #
coverage
```

22. user (2026-06-18T02:07:13.682Z)

```
backend\pipeline\validators\__init__.py          14      0  100%
backend\pipeline\validators\base.py              39      3   92%
backend\providers\__init__.py                   16      1   94%
backend\providers\base.py                       9      1   89%
backend\providers\gemini.py                     98     19   81%
backend\reporting\__init__.py                   26      2   92%
backend\reporting\harvest.py                   111      8   93%
backend\reporting\run_signals.py                14      0  100%
backend\results.py                             27      0  100%
backend\sports\__init__.py                     17      1   94%
backend\sports\badminton.py                    13      1   92%
backend\sports\base.py                         12      3   75%
backend\tools\__init__.py                      0      0  100%
backend\tools\export.py                        75      2   97%
backend\tools\rally_slicer.py                  104     47   55%
backend\utils\__init__.py                      0      0  100%
backend\utils\geometry.py                      29      0  100%
backend\utils\hashing.py                       9      0  100%
backend\utils\logging_setup.py                 11      0  100%
backend\utils\paths.py                         27      3   89%
backend\utils\run_telemetry.py                 126     12   90%
backend\utils\video.py                         25      3   88%
training\__init__.py                           0      0  100%
training\gen0\__init__.py                      0      0  100%
training\gen0\harness.py                       108     11   90%
-----
TOTAL                                           7990   1521   81%
Required test coverage of 70% reached. Total coverage: 80.96%
===== short test summary info =====
FAILED tests/test_served_gate_a_regression.py::test_served_gate_a_does_not_regress_f1
FAILED tests/test_served_gate_a_regression.py::test_proposal_windowing_reproduces_offline_lift
2 failed, 855 passed, 7 warnings in 105.13s (0:01:45)
=====
RESULT: FAIL - pytest exit 1
=====
```

23. assistant (2026-06-18T02:07:17.679Z)

Coverage is 80.96% (above the 70% floor), and `backend/results.py` is 100% covered. But there are 2 test failures: `test\_served\_gate\_a\_regression.py`. The reviews claimed only a **different** pre-existing failure (`test\_fusion\_hybrid::test\_gate\_a\_never\_emptyies\_a\_video`). I need to

determine whether these failures are (a) pre-existing on master and (b) unrelated to this PR's touched files. This is exactly the kind of claim I must not trust ? let me verify directly.

24. user (2026-06-18T02:07:33.732Z)

```
fps, fw = float(s["fps"]), float(s["frame_width"])
pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
wins = TrackNetRunner.trajectory_to_action_windows(
    pts, fps=fps, frame_width=fw, velocity_thresh=_PROPOSAL[0],
    merge_gap=_PROPOSAL[1], min_window_duration=_PROPOSAL[2])
iv = [(w["start"], w["end"]) for w in wins]
est = rally_gate.estimate_play_axis(pts)
gated = rally_gate.gate_windows(pts, iv, fps, min_crossings=0, estimate=est)
on = [(g.start, g.end) for g in gated if g.crossings >= 2]
gts = load_golden_gts(s["labels"])
d_on.append(score(on, gts).f1)
d_off.append(score(iv, gts).f1)
seg_on.append(_segm.segment_count_ratio(on, gts))
seg_off.append(_segm.segment_count_ratio(iv, gts))
n = len(d_on)
mean_delta = (sum(d_on) - sum(d_off)) / n
wins = sum(1 for a, b in zip(d_on, d_off) if a > b + 1e-9)
> assert mean_delta == pytest.approx(0.074, abs=0.01), mean_delta
E AssertionError: -0.03652587196942281
E assert -0.03652587196942281 == 0.074 ± 0.01
E
E comparison failed
E Obtained: -0.03652587196942281
E Expected: 0.074 ± 0.01

tests\test_served_gate_a_regression.py:157: AssertionError
===== short test summary info =====
FAILED tests/test_served_gate_a_regression.py::test_served_gate_a_does_not_regress_f1
FAILED tests/test_served_gate_a_regression.py::test_proposal_windowing_reproduces_offline_lift
2 failed, 3 passed in 13.63s
```

25. assistant (2026-06-18T02:07:44.902Z)

This failure has nothing to do with the PR ? it's a numeric regression assertion in the served-gate-A eval (a golden-fixture F1 comparison). The PR touches none of `rally\_gate.py`, `windowing.py`, `served\_gate\_a.py`, or the trajectory CSV fixtures. To confirm it's pre-existing, let me run these tests against `origin/master` (clean of this PR).

26. user (2026-06-18T02:07:59.803Z)

```
=ON MASTER=
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
=== run failing tests on master ===

tests\test_served_gate_a_regression.py:157: AssertionError
===== short test summary info =====
FAILED tests/test_served_gate_a_regression.py::test_served_gate_a_does_not_regress_f1
FAILED tests/test_served_gate_a_regression.py::test_proposal_windowing_reproduces_offline_lift
2 failed, 3 passed in 11.09s
```

27. assistant (2026-06-18T02:08:05.532Z)

Confirmed pre-existing on `origin/master`: both `test\_served\_gate\_a\_regression.py` failures reproduce identically without this PR. They are a golden-corpus numeric drift in code this PR does not touch ? not introduced by PR #213, and orthogonal to the unwrap fix. (The reviewers named a different pre-existing failure; the actual local failures are these two, but the conclusion ? pre-existing, unrelated ? holds and I verified it directly rather than trusting the claim.)

Let me restore the PR branch state and do a final confirm of the touched-files behavior end to end.

28. user (2026-06-18T02:08:09.874Z)

=BACK ON PR=
all edge cases OK

29. assistant (2026-06-18T02:08:12.709Z)

All edge cases verified at runtime, including the ``Success(value=[])` ? `([], None)` path that feeds the empty-guard, and the `Success(value=None)` ? `(None, None)` corner (correct: the empty-guard catches it).`

Let me restore the working branch to where it started.

30. user (2026-06-18T02:08:13.776Z)

=restored=
docs/golden-regression-fixtures-plan

31. assistant (2026-06-18T02:08:26.079Z)

Verification complete. Summary of what I confirmed independently (not trusting the review claims):

****Bug premise ? REAL.**** ``backend/pipeline/segmenters/motion.py`` returns ``Failure`` (line 31) / ``Success(value=...)`` (line 224). The two scripts previously did ``len()``/truthiness on that raw dataclass, which has no ``__len__`` ? ``TypeError``. ``run_phase3_eval.py``'s ``--segmenter`` default is literally ``"motion"`` (line 54), so it crashed on the default path. ``run_process_and_stitch.py`` uses ``cfg.indexing.default_segmenter``, whose config default is ``"yolo_hybrid"`` (bare list) ? so its crash is ****latent****, only when configured with motion. The inline comment ("for some segmenters (e.g. ``motion``)") is accurate; only the commit subject "motion crashes the default path" overgeneralizes for the second script. Prose nit, not a defect.

****Fix ? FAITHFUL.**** ``unwrap_or_failure`` exactly mirrors ``backend/main.py``'s inline ``isinstance(result, Failure)`` + ``result.value if hasattr(result, "value") else result`` (lines 201/213 and 838/843, both untouched and still passing). Failure branching is correct: ``continue`` in the batch driver, ``return`` in the single-video script.

****Edge cases ? verified at runtime.**** ``Success(value=[])``?``([],None)`` flows to the empty-guard; ``Failure``?``(None,f)``; bare list passes through; ``Success(value=None)``?``(None,None)`` is caught by the empty-guard.

****No other latent caller.**** ``tools/generate_highlights.py:168`` handles ``Failure`` and reads segments from the DB (never ``len()``'s the result) ? correct, a 4th hand-rolled unwrap (DRY nit, out of scope).

****Gate checks.**** New tests 4/4 pass; ``backend/results.py`` 100% covered; ruff clean; mypy clean on the CI scope (``backend/training``, exit 0 ? ``scripts/`` is not type-gated). Full suite: 855 passed, coverage 80.96% (?70% floor).

****Test failures are PRE-EXISTING and UNRELATED.**** The 2 local failures (``test_served_gate_a_regression.py``) reproduce identically on ``origin/master`` ? a golden-corpus numeric drift in ``rally_gate``/``windowing`` code this PR does not touch. Not introduced by #213.

No genuine must-fix exists. Verdict: merge.

32. user (2026-06-18T02:08:36.738Z)

Structured output provided successfully

33. assistant (2026-06-18T02:08:45.203Z)

Verdict: ****merge****. The fix is correct, faithful to the existing inline unwrap, fully verified locally (new tests pass, ruff/mypy clean on CI scope, full suite 855 passed at 80.96% coverage), and the 2 local test failures are pre-existing on master in untouched code. No must-fixes; only three minor non-blocking nits (prose accuracy, optional DRY follow-up, optional empty-success test assertion).